

BudgetMate

Weekly Mini Project 02 — Team 2 · Project Specification

Project	BudgetMate
Team leader	Yazan Shartouh
Team members	Hamid Al Haj, Esmaeel Mahfoud, Mulham Al Kasir, Nagham Jaza, Alaa Ismaiel

Know where your money goes — every month, every category, at a glance.

1. Project Overview

Many people know roughly how much they earn, but not where their money actually goes. By the end of the month the money is gone and nobody can say whether food, transport or entertainment was the reason.

BudgetMate is a personal finance application. The user records income and expenses, sets a monthly budget for each spending category, and instantly sees their financial position: how much came in, how much went out, how much was saved, and which categories are over budget.

2. Team

The team leader and members are listed in the table at the top of this document.

Team Responsibility

- The leader coordinates the internal work split and makes sure the team reaches a working integrated result.
- Frontend/backend ownership is not assigned by the instructor.
- The team should decide how responsibilities are distributed.
- Every member must make a meaningful contribution.
- Every member should understand the project at a high level.
- Every member must participate in the final English presentation.

3. Project Goal

Build a web application where a single user can:

- record income and expense transactions;
- organize transactions into categories;
- browse, filter, edit and delete transactions;
- set a monthly budget for each expense category;
- see a dashboard and a monthly report that summarize their financial position correctly.

All totals, balances and budget statuses must always be calculated from the recorded application data and remain consistent after every change.

4. Technology Constraints

Area	Requirement
Frontend	React + TypeScript
Backend	NestJS

Area	Requirement
Data	In-memory only
Database	Not required

- The application's working data is kept in memory and may reset when the application server restarts.
- How you organize your application is your team's decision.
- Do **not** add a database, user accounts/login, external AI services, cloud services or paid APIs.

5. User Scenario

Omar receives his salary of 1,200 on the 1st of the month and does some freelance work worth 300 in the middle of the month. He sets monthly budgets: Food 300, Transport 100, Entertainment 80.

During the month he records his expenses as they happen. When he adds a 45 cinema ticket that pushes Entertainment to 95, the Budgets screen marks Entertainment as **Exceeded by 15**. On the Dashboard he sees his total income (1,500), his total expenses, his remaining balance, and that Food is 62% of all his spending.

At the end of the month he opens the Monthly Summary, selects the month, and sees that he saved 410 — 27% of his income — and that Food was his largest category.

6. Required Features

Feature	Description
Record income	Add an income transaction.
Record expense	Add an expense transaction.
Categories	Every transaction belongs to a category. Income and expense have separate category lists.
Transaction history	View all transactions, newest first.
Filter transactions	Filter by type (income / expense), category and month. Filters can be combined.
Edit and delete	Correct or remove an existing transaction.
Monthly budgets	Set, change or remove a budget for an expense category in a specific month.
Dashboard	A summary of the user's current financial position.
Budget usage	See, for each budgeted category, how much has been spent and how much remains.
Monthly summary	A report for any chosen month.

Transaction information

Each transaction must present at least:

- type (income or expense)
- amount
- category
- date
- a short description / note (optional for the user)

Categories

The application must offer a fixed set of categories prepared by your team — at least **6 expense categories** (for example: Food, Transport, Housing, Bills, Health, Entertainment) and at least **3 income categories** (for example: Salary, Freelance, Gift). Creating new categories is not required.

7. Business Rules

Transactions

1. An amount must be a valid **positive** number greater than zero, with at most two decimal places.
2. Income **increases** the available balance; an expense **decreases** it.
3. An income transaction can only use an income category; an expense transaction can only use an expense category.
4. A transaction date is required and cannot be in the future.
5. Editing or deleting a transaction must immediately be reflected in every total, balance, budget and report that includes it.

Totals and balance

1. All totals are calculated from the recorded transactions — they are never typed in by the user.
2. The **remaining balance** is all income minus all expenses, across all recorded transactions.
3. A **monthly** total includes only the transactions dated in that month.
4. **Monthly savings = that month's income – that month's expenses.** If expenses are larger than income, the savings amount is negative and the month must be clearly shown as **overspent**.
5. A category's total is the sum of its transactions for the period being shown.

Budgets

1. A budget belongs to one expense category and one month. A category can have at most one budget per month.
2. A budget amount must be a positive number. Budgets cannot be set for income categories.
3. Each budgeted category shows a status:
 - **On track** — less than 80% of the budget has been spent;
 - **Near limit** — between 80% and 100% of the budget has been spent;
 - **Exceeded** — more than 100% of the budget has been spent, together with the amount by which it was exceeded.
4. Recording an expense that exceeds a budget is **allowed** (it is real spending), but the category must then show as Exceeded.
5. Expense categories without a budget are still counted in all totals; they are simply shown as "No budget".

8. Required Screens / Views

Screen	Must show / allow
Dashboard	For the current month: total income, total expenses, savings and savings rate; the overall remaining balance; spending by category with percentages; a list of categories that are near their limit or exceeded; the most recent transactions.
Transactions	The full transaction history with filters; the ability to edit and delete each transaction. A clear message when no transactions match.
Add Transaction	A form to record income or an expense. The same form (or a similar one) is used to edit an existing transaction.
Budgets	For a chosen month: each expense category with its budget, amount spent, amount remaining or exceeded, percentage used and status; the ability to set, change and remove budgets.
Monthly Summary / Report	For a chosen month: total income, total expenses, savings, savings rate, spending per category with percentage, budget comparison per category, and the largest spending category.

All screens must be reachable through the application's navigation.

9. Required User Flows

Flow A — Record a transaction

1. The user opens Add Transaction.
2. The user chooses income or expense, then enters amount, category, date and an optional note.
3. If any information is invalid, the transaction is rejected and the user sees why.
4. If valid, the transaction is saved and appears in the transaction history.
5. The Dashboard, Budgets and Monthly Summary all reflect the new transaction.

Flow B — Correct a mistake

1. On the Transactions screen, the user finds a transaction with a wrong amount.
2. The user edits the amount and saves.
3. Every total, balance and budget status that includes this transaction updates.
4. The user deletes a duplicate transaction, and the totals update again.

Flow C — Plan and follow a budget

1. On the Budgets screen, the user selects a month and sets a budget for several expense categories.
2. The user records expenses in those categories.
3. Each category shows how much is spent, how much remains and its status.
4. When an expense pushes a category over its budget, the category shows **Exceeded** and the amount it is over by.

Flow D — Review a month

1. The user opens the Monthly Summary and selects a month.
2. The report shows only that month's transactions in its totals.
3. The user sees the savings amount, the savings rate, spending per category and the budget comparison.
4. The user selects a different month and every value changes to that month.

10. Required Calculations / Derived Information

The application must produce and display:

Derived information	Where it appears
Total income (current month / selected month)	Dashboard, Monthly Summary
Total expenses (current month / selected month)	Dashboard, Monthly Summary
Remaining balance (all time)	Dashboard
Savings amount (income – expenses) for a month	Dashboard, Monthly Summary
Savings rate (savings as a percentage of income)	Dashboard, Monthly Summary
Spending per category and its percentage of total spending	Dashboard, Monthly Summary
Budget used (amount and percentage) per category	Budgets, Monthly Summary
Budget remaining, or amount exceeded, per category	Budgets, Monthly Summary
Budget status (On track / Near limit / Exceeded / No budget)	Budgets, Dashboard
Largest spending category of the month	Monthly Summary

When a month has no income, the savings rate must be shown as not available — never as an error or a meaningless number.

11. Validation & Error Scenarios

General expectations

- Required information cannot be missing.
- Numeric values must be sensible (no negative, zero or absurd values where they make no sense).
- Invalid actions must be rejected — the system must never silently accept an invalid operation.
- The user must always receive a clear, human-readable message explaining **why** an action failed.
- Invalid operations must be rejected even if the user reaches them through an unexpected interaction (for example, an outdated page or a repeated submission).
- When a request cannot be completed or something goes wrong, the screen must show a sensible message — never a blank page or a frozen screen.

Project-specific scenarios

Situation	Expected behavior
Amount is empty, zero, negative, not a number, or has more than two decimals	Rejected with a clear message.
Category is missing, does not exist, or does not match the transaction type	Rejected with a clear message.
Date is missing, invalid or in the future	Rejected with a clear message.
Editing or deleting a transaction that does not exist	A clear "transaction not found" message.
Budget amount is zero, negative or not a number	Rejected with a clear message.
A budget is set for an income category	Rejected with a clear message.
A second budget is created for the same category and month	Rejected (or treated clearly as an update of the existing budget — your team decides, but it must never create two budgets).
A month with no transactions is selected	The summary shows zero values and a clear "no transactions this month" message.

12. In-Memory Data Requirements

No database is required.

- The application may start with seeded (pre-loaded) data.
- Data created or changed while the application is running must behave normally during that run — every screen must reflect the latest data, including after a page refresh.
- It is acceptable that all data resets when the application server restarts. This is an expected limitation of this assignment — do not spend time adding any other persistence technology.

For this project specifically:

- The application must start with the fixed category lists and at least **20 realistic transactions** spread over at least **2 different months**, using several categories.
- The seeded data must include budgets for the current month in which at least one category is **On track**, one is **Near limit** and one is **Exceeded** — so every status can be demonstrated.
- There is a single user; no user selection or login is needed.

13. Out of Scope

The following are **not required** and should not be built as part of the core project: bank or payment integrations; user accounts, login or registration; currency conversion or currency APIs (one currency is enough); financial forecasting; a database; charting libraries (charts are a bonus only).

14. Minimum Deliverable

Your project counts as complete only when **all** of the following work:

- ☐ A working React + TypeScript frontend.
- ☐ A working NestJS backend.
- ☐ The frontend and backend work together as one integrated application; every change a user makes is visible on all related screens.
- ☐ Every required screen in section 8 exists and is reachable through the application's navigation.
- ☐ Every business rule in section 7 is enforced.
- ☐ Invalid operations are rejected with useful, clear feedback.
- ☐ In-memory data changes correctly while the application is running.
- ☐ The project can be demonstrated from start to finish without manual fixes during the demo.
- ☐ Income and expenses can be added, edited and deleted.
- ☐ Transactions can be filtered by type, category and month (combined).
- ☐ Budgets can be set, changed and removed per category per month.
- ☐ Dashboard and Monthly Summary values are correct and update after every change.
- ☐ Budget statuses (On track / Near limit / Exceeded) are correct.

15. Optional Bonus Features

Bonus features are **optional**. They are not needed for full core completion and they never replace a missing core requirement. Only start a bonus after the minimum deliverable is complete and working. Bonus features must not add a database, login or new major technologies.

- **Simple charts** — a visual breakdown of spending by category.
- **Month comparison** — compare two months side by side (income, expenses, savings, categories).
- **Category colors / icons** — give each category a recognizable color or icon used across all screens.
- **Responsive dashboard** — a dashboard that works well on a phone-sized screen.

16. Presentation Expectations

Your team will present the finished project **in English**. Your instructor will confirm the date and the time limit.

The presentation must include:

1. **The problem** — a brief explanation of who the product is for and what problem it solves.
2. **Product walkthrough** — a short tour of the main screens.
3. **Live demonstration** — the main user flows working live, including at least one invalid case being rejected with a clear message.
4. **Business rules** — an explanation, in your own words, of the most important rules your product enforces and how a user experiences them.
5. **Team** — every member speaks.

Before presenting, make sure the backend and frontend both start cleanly and the demo data is ready.

17. Evaluation Focus

You will be assessed on:

Area	What we look for
Functional completeness	The required features and screens exist and work.
Correct business behavior	The business rules are enforced exactly as described.
Frontend/backend integration	The frontend and backend genuinely work together as one working application.
User experience	Screens are clear, readable and easy to use without explanation.
Handling invalid cases	Invalid input and invalid actions are rejected with clear messages.
Team collaboration	Work is shared; every member contributed meaningfully.
Code organization and readability	Your code is understandable and reasonably organized.
Explaining your own work	Every member can explain the product and its rules.
Presentation quality	Clear English presentation with a working live demo.